

ABC457 F - Second Gap 题解

题意概括

给定长度为 $N - 1$ 的数组 D ，要求统计有多少个排列 P 满足：

- 对每个后缀 $(P_i, P_{i+1}, \dots, P_N)$,
- 其中最大值和第二大值所在位置的距离恰好是 D_i 。

答案对 998244353 取模。

解题思路

1. 从后往前做“插入 DP”

考虑从后缀开始构造排列。

假设我们已经处理好了后缀 $(P_{i+1}, \dots, P_N)$ ，现在要在最前面插入一个新元素 P_i 。这里只关心相对大小，不关心具体数字，这和经典的插入 DP 是一样的想法。

对于当前后缀，最重要的是：

- 最大值在哪个位置；
- 第二大值在哪个位置。

如果把这两个位置记成 $\{a, b\}$ ，那么插入 P_i 后有三种情况：

1. P_i 成为新的最大值
此时新的前两大位置变成 $\{i, a\}$ 。
2. P_i 成为新的第二大值
此时新的前两大位置变成 $\{a, i\}$ 。
3. P_i 既不是最大值，也不是第二大值
此时新的前两大位置仍然是 $\{a, b\}$ 。

2. 为什么只记录“最大值的位置”就够了，而且可以直接开数组

朴素想法会想记录 (a, b) 两个位置，但其实没必要。

原因是：

- 在前两种转移里，只要知道原来的最大值位置 a ，就已经能确定是否合法；
- 在第三种转移里，前两大位置根本不变，因此要想继续满足条件，只要求 $D_i = D_{i+1}$ 。

也就是说，第三种转移完全不依赖第二大值的位置 b 的具体数值。

因此可以把状态压缩成：

- $dp[i][a]$ ：表示后缀 $(P_i, \dots, P_N)$ 中，最大值出现在位置 a 的方案数。

这里的状态下标本来就是“位置”，范围固定在 $1 \sim N$ ，所以没有必要使用哈希表或映射。

我们直接开普通数组， $dp[a]$ 表示位置 a 这个状态的值即可。

3. 转移怎么写

为了实现方便，下面用一维 $dp[a]$ 表示当前层的状态。

初始化看长度为 2 的后缀，也就是 (P_{N-1}, P_N) ：

- 最大值可能在 $N - 1$
- 也可能在 N

所以：

- $dp[N - 1] = 1$
- $dp[N] = 1$

接下来从 $i = N - 2$ 倒着做到 1。

设：

- $j = i + D_i$

因为无论 P_i 成为最大值还是第二大值，原来后缀中的最大值位置都必须与 i 的距离等于 D_i ，所以那个位置只能是 j 。

于是：

1. 若 P_i 成为最大值

新的最大值位置变成 i ，因此：

- 从原来最大值在 j 的状态，转移到新的 $dp[i]$

2. 若 P_i 成为第二大值

新的最大值位置仍然是 j ，因此：

- 从原来最大值在 j 的状态，转移到新的 $dp[j]$

3. 若 P_i 既不是最大值也不是第二大值

只有当 $D_i = D_{i+1}$ 时才可能，因为这时“前两大位置的距离”不变。

此外，这种插入方式有：

◦ $N - i - 1$

种选择，因为当前后缀长度为 $N - i$ ，除去排在前两位的可能外，还可以插入到其余相对等级中。

4. 如何把总复杂度优化到 $O(N)$

如果第三种转移每次都对所有状态乘上同一个常数，那么朴素写法会是 $O(N^2)$ 。

但注意到：

- 第三种转移只是“把所有状态整体乘一个相同的系数”

所以可以把这个系数单独记在外面，设为 `coef`：

- `dp[a]` 只存“去掉公共系数后的值”
- 真正的状态值等于 `dp[a] * coef`

这样：

- 第三种转移只需要更新 `coef`
- 第一、二种转移只对两个位置做单点修改

另外，当 $D_i \neq D_{i+1}$ 时，旧状态里只有位置 $j = i + D_i$ 的值还会被用到，其余状态都会整体失效。如果每次都把整个 `dp` 数组清零，也会退化成 $O(N^2)$ 。

这时可以再配一个 `mark[a]` 数组和当前轮编号 `tag`：

- `mark[a] = tag` 表示位置 `a` 这个状态在当前这一轮是有效的；
- 需要“清空整张表”时，只要把 `tag` 加一；
- 之后重新写入这轮真正会保留下来的少数状态即可。

由于状态下标本来就是 $1 \sim N$ ，这种“数组 + 轮次标记 + 公共系数”的写法就足够了，不需要 `unordered_map`。

正确性说明

状态定义正确

我们定义 `dp[a]` 表示当前处理到某个后缀时，“最大值出现在位置 a ”的方案数。

由于题目在每一层只关心最大值和第二大值之间的距离，而第二大值的位置在前两种转移中不需要单独知道，在第三种转移中又只会保持不变，因此只记录最大值位置就足够了。

又因为这个位置一定落在 $1 \sim N$ 内，所以直接用普通数组按位置存状态，就是最自然的实现方式。

转移一、二正确

当插入的新元素 P_i 成为最大值时，原来的最大值位置必须距离 i 恰好为 D_i ，所以原来的最大值只能在 $j = i + D_i$ 。此时新状态的最大值位置变成 i ，因此可以从旧状态 $dp[j]$ 转移到新状态 $dp[i]$ 。

当插入的新元素 P_i 成为第二大值时，原来的最大值位置同样必须是 $j = i + D_i$ ，并且新状态的最大值位置仍然是 j ，因此可以从旧状态 $dp[j]$ 转移到新状态 $dp[j]$ 。

转移三正确

当 P_i 既不是最大值也不是第二大值时，后缀中的最大值位置和第二大值位置都不发生变化，所以它们之间的距离也不变。因此这种转移可行的充要条件是：

- $D_i = D_{i+1}$

一旦这个条件满足，当前元素可以放在除前两名之外的任意相对名次，总共有 $N - i - 1$ 种可能，因此所有状态都统一乘上这个系数。

懒乘系数正确

第三种转移对所有状态都乘同一个值，因此把它抽成公共因子 `coef` 不会改变任何状态之间的相对关系，也不会影响第一、二种转移的正确性。最终再把所有状态和乘回 `coef`，就能得到真实答案。

轮次标记正确

当 $D_i \neq D_{i+1}$ 时，新一层只会保留由旧状态中位置 $j = i + D_i$ 转移出来的两个位置，其余位置全部失效。

此时把 `tag` 加一，相当于把旧轮次的状态整体作废；只有重新写入并标记为当前 `tag` 的位置，才会在之后继续参与转移。

因此，`mark[a]` 只是在实现上避免整张数组反复清零，不会改变 DP 本身的状态集合，所以这种优化是正确的。

综上，DP 的状态、转移和优化都正确，因此算法正确。

复杂度

- 时间复杂度： $O(N)$
- 空间复杂度： $O(N)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;
```

```
const int MAXN = 200000 + 5;
const long long MOD = 998244353;

int d[MAXN];
long long inv_num[MAXN];
long long dp[MAXN];
int mark[MAXN];
int cur_tag;

// 如果当前位置不是当前这一轮的有效状态，就返回 0
long long get_value(int pos){

    if(mark[pos] != cur_tag){
        return 0;
    }
    return dp[pos];
}

// 给当前位置加上一个状态值
void add_value(int pos, long long val){

    if(val == 0){
        return;
    }

    if(mark[pos] != cur_tag){
        mark[pos] = cur_tag;
        dp[pos] = 0;
    }

    dp[pos] += val;
    if(dp[pos] ≥ MOD){
        dp[pos] -= MOD;
    }
}

int main(){

    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

// 读取排列长度
int n;
cin >> n;

// d[i] 表示第 i 层要求的距离
for(int i=1; i≤n - 1; i++){
    cin >> d[i];
}

// 预处理逆元, 后面需要把单点新增量换算到新的公共系数下
inv_num[1] = 1;
for(int i=2; i≤n; i++){
    inv_num[i] = (MOD - MOD / i) * inv_num[MOD % i] % MOD;
}

// 初始长度为 2 的后缀, 最大值可以在 n - 1 或 n
cur_tag = 1;
add_value(n - 1, 1);
add_value(n, 1);

// coef 记录所有状态共同乘上的系数
long long coef = 1;

for(int i=n - 2; i≥1; i--){
    int j = i + d[i];
    long long v = get_value(j);

    if(d[i] == d[i + 1]){
        // 第三种转移: 所有状态统一乘上 ways
        long long ways = n - i - 1;
        coef = coef * ways % MOD;

        // 把“成为最大值 / 第二大值”的新增量换算到新的公共系数下
        long long add = v * inv_num[ways] % MOD;
        add_value(j, add);
        add_value(i, add);
    } else {
        // 第三种转移不存在, 上一轮状态整体作废
        cur_tag++;
        add_value(j, v);
        add_value(i, v);
    }
}

```

```
    }  
}  
  
// 把去掉的公共系数乘回去  
long long sum = 0;  
for(int i=1; i≤n; i++){  
    if(mark[i] == cur_tag){  
        sum += dp[i];  
        if(sum ≥ MOD){  
            sum -= MOD;  
        }  
    }  
}  
  
long long ans = sum * coef % MOD;  
cout << ans;  
  
return 0;  
}
```